

DigiKhata Style Bookkeeping App — Full Project Specification

Project Name

SmartKhata

Overview

Create a modern bookkeeping and debt management application similar to DigiKhata, Khatabook, and OkCredit.

The app should help shopkeepers and small businesses manage:

- Customer debts
- Payments
- Expenses
- Income
- Sales
- Reports
- Receipts
- Analytics

The application must support:

- Android
 - iOS
 - Web
-

Recommended Tech Stack

Frontend

- React Native (Expo)
- TypeScript
- NativeWind / Tailwind CSS
- React Navigation
- React Query
- Zustand or Redux Toolkit

Backend

- Firebase Authentication
- Firebase Firestore
- Firebase Storage
- Firebase Cloud Messaging

Optional Backend

- Node.js + Express API
-

Folder Structure

```
src/  
├─ components/  
├─ screens/  
├─ navigation/  
├─ services/  
├─ firebase/  
├─ hooks/  
├─ store/  
├─ utils/  
├─ types/  
├─ assets/  
├─ constants/  
└─ theme/
```

Authentication Module

Features

- Phone OTP Login
- Email & Password Login
- Google Login
- Forgot Password
- User Profile
- Logout

Firestore Auth Integration

Required screens:

1. Splash Screen
 2. Login Screen
 3. Register Screen
 4. OTP Verification Screen
 5. Forgot Password Screen
-

Dashboard Module

Dashboard Cards

- Total Customers
- Total Credit
- Total Paid
- Remaining Balance
- Monthly Revenue
- Monthly Expenses

Charts

- Weekly income chart
- Expense chart
- Debt collection chart

Quick Actions

- Add Customer
 - Add Debt
 - Record Payment
 - Add Expense
-

Customer Management

Features

- Add customer
- Edit customer
- Delete customer
- Customer details page

- Upload customer image
- Customer notes
- WhatsApp shortcut
- Call shortcut

Customer Fields

```
interface Customer {  
  id: string;  
  name: string;  
  phone: string;  
  address?: string;  
  image?: string;  
  notes?: string;  
  createdAt: Timestamp;  
}
```

Debt Management Module

Features

- Add debt entry
- Partial payment support
- Remaining balance auto calculation
- Due date tracking
- Payment history
- Settlement tracking
- Printable receipts

Debt Model

```
interface Debt {  
  id: string;  
  customerId: string;  
  totalAmount: number;  
  paidAmount: number;  
  remainingAmount: number;  
  dueDate?: string;  
  notes?: string;  
  status: 'paid' | 'unpaid' | 'partial';  
  createdAt: Timestamp;  
}
```

Payment Module

Features

- Add payment
- Payment history
- Generate receipt
- Share receipt
- Print receipt

Payment Model

```
interface Payment {  
  id: string;  
  debtId: string;  
  customerId: string;  
  amount: number;  
  paymentMethod: string;  
  notes?: string;  
  createdAt: Timestamp;  
}
```

Expense Management

Features

- Add expense
- Expense categories
- Daily expense report
- Monthly expense analytics

Expense Categories

- Rent
 - Electricity
 - Internet
 - Staff Salary
 - Fuel
 - Miscellaneous
-

Reports Module

Reports

- Daily report
- Weekly report
- Monthly report
- Customer statement
- Profit & Loss
- Credit report
- Expense report

Export Options

- PDF Export
 - Excel Export
 - Print Support
-

Notifications

Firestore Cloud Messaging

Notification Types

- Payment reminders
 - Due date alerts
 - Daily summaries
 - Monthly summaries
-

Offline Support

Features

- Local cache
 - Offline entry support
 - Auto sync when internet reconnects
-

Multi-language Support

Languages:

- English
 - Pashto
 - Dari
 - Urdu
 - Arabic
-

Dark Mode

Theme Support

- Light Theme
 - Dark Theme
 - System Theme
-

Security

Firebase Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    match /customers/{customerId} {
      allow read, write: if request.auth != null;
    }

    match /debts/{debtId} {
      allow read, write: if request.auth != null;
    }

    match /payments/{paymentId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

```
}  
}
```

UI/UX Requirements

Design Style

- Minimal
- Professional
- Modern fintech style
- Mobile-first
- Fast loading
- Smooth animations

Navigation

- Bottom tabs
- Drawer menu

Screens

1. Splash
2. Login
3. Register
4. Dashboard
5. Customers
6. Customer Details
7. Add Debt
8. Payments
9. Reports
10. Settings
11. Notifications
12. Profile

Settings Module

Features

- Language selection
- Theme toggle
- Backup & Restore

- Notification settings
 - Currency selection
-

Admin Panel

Features

- User management
 - Analytics
 - Revenue tracking
 - Subscription management
 - App settings
-

Firebase Collections

```
users/  
customers/  
debts/  
payments/  
expenses/  
notifications/  
reports/
```

Required Packages

React Native Packages

```
npm install firebase  
npm install react-navigation  
npm install react-native-paper  
npm install nativewind  
npm install react-native-chart-kit  
npm install react-native-reanimated  
npm install react-native-svg  
npm install zustand  
npm install react-query
```

```
npm install react-hook-form  
npm install yup
```

App Flow

```
graph TD  
  A[Splash Screen] --> B[Authentication]  
  B --> C[Dashboard]  
  C --> D[Customers]  
  D --> E[Debts]  
  E --> F[Payments]  
  F --> G[Reports]
```

Performance Requirements

- Optimized for low-end Android phones
- Fast Firestore queries
- Lazy loading
- Pagination
- Image optimization
- Real-time synchronization

Deliverables

The engineer must provide:

- Full source code
- Clean architecture
- Reusable components
- Proper documentation
- APK build
- iOS build
- Web deployment

- Firebase setup
 - Admin panel
 - Production-ready code
-

Bonus Features

- QR payment support
 - Barcode scanner
 - Voice notes
 - AI expense categorization
 - Multi-shop support
 - Team member accounts
 - Inventory management
 - POS system integration
-

Final Goal

The final app should be:

- Production-ready
- Scalable
- Secure
- Modern
- Fast
- Similar to DigiKhata
- Easy for small businesses to use
- Optimized for Afghanistan, Pakistan, and India markets